Singly Linked List – GetNthNodeFromEnd2 and Time Complexity.

[*Nth Node from end of a Linked List*, 0 *based indexing*]

This function helps us to get the last node from the end of linked list.

Program :

```
int getNthNodeFromEnd2(Node **head, int node)
{
    int len = 0;
    Node *current;
    current = *head;
    while (current != nullptr)
    {
        current = current->next;
        len++;
    }
    if (node < 0 || node >= len)
    {
        cout << "Error: Invalid index." << endl;
        return -1;
    }
    else
    {
        current = *head;
        for (int i = 0; i < len - node - 1; i++)
        {
            current = current->next;
        }
        return current->data;
    }
}
```

```
case 21:
        {
            cout << endl;
            cout << endl;
            cout << "Nth node from end , where 0 is the last node(0 based
indexing):" << endl;
            int n;
            cout << "Enter the value of node: ";
            cin >> n;
            cout << "The " << n << "th node from the end is: " <<
getNthNodeFromEnd2(&head, n) << endl;
            cout << endl;
            cout << endl;
            break;
        }
```

Program Analysis:

Function Call:

$Function\ call\ occurs\ when\ Case\ 21\ is\ initiated\ from\ Switch - Case$:

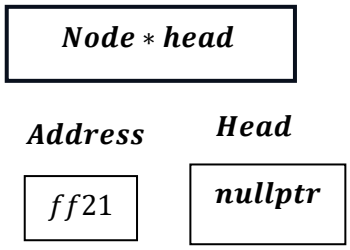
$cout \ldots\ldots getNthNodeFromEnd2(\&head, n);$

$There\ are\ two\ criteria\ of\ input\ Linked\ List$:

Avinandan

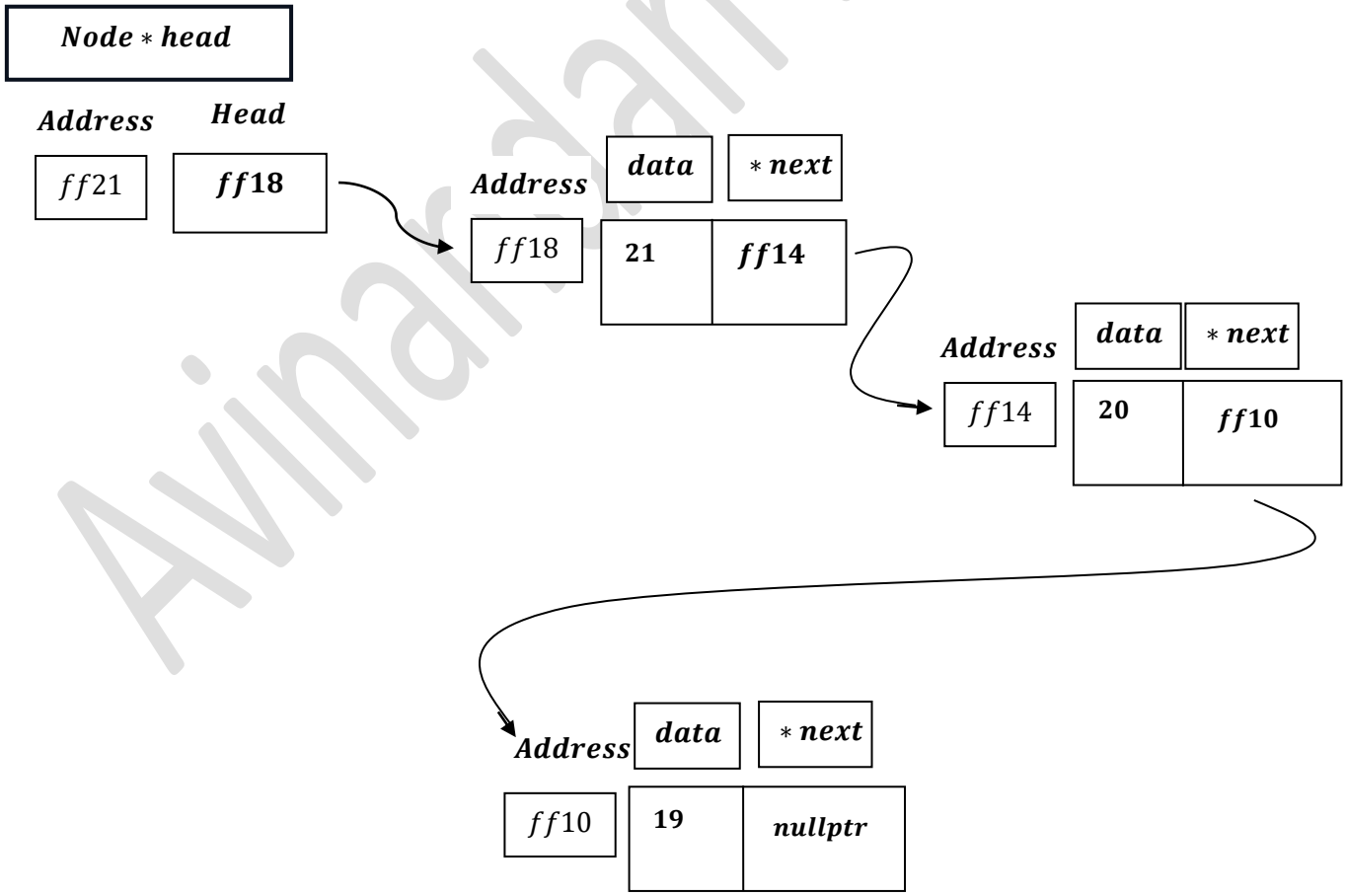
1) *Linked List is Empty*:

Pictorially depicted as:



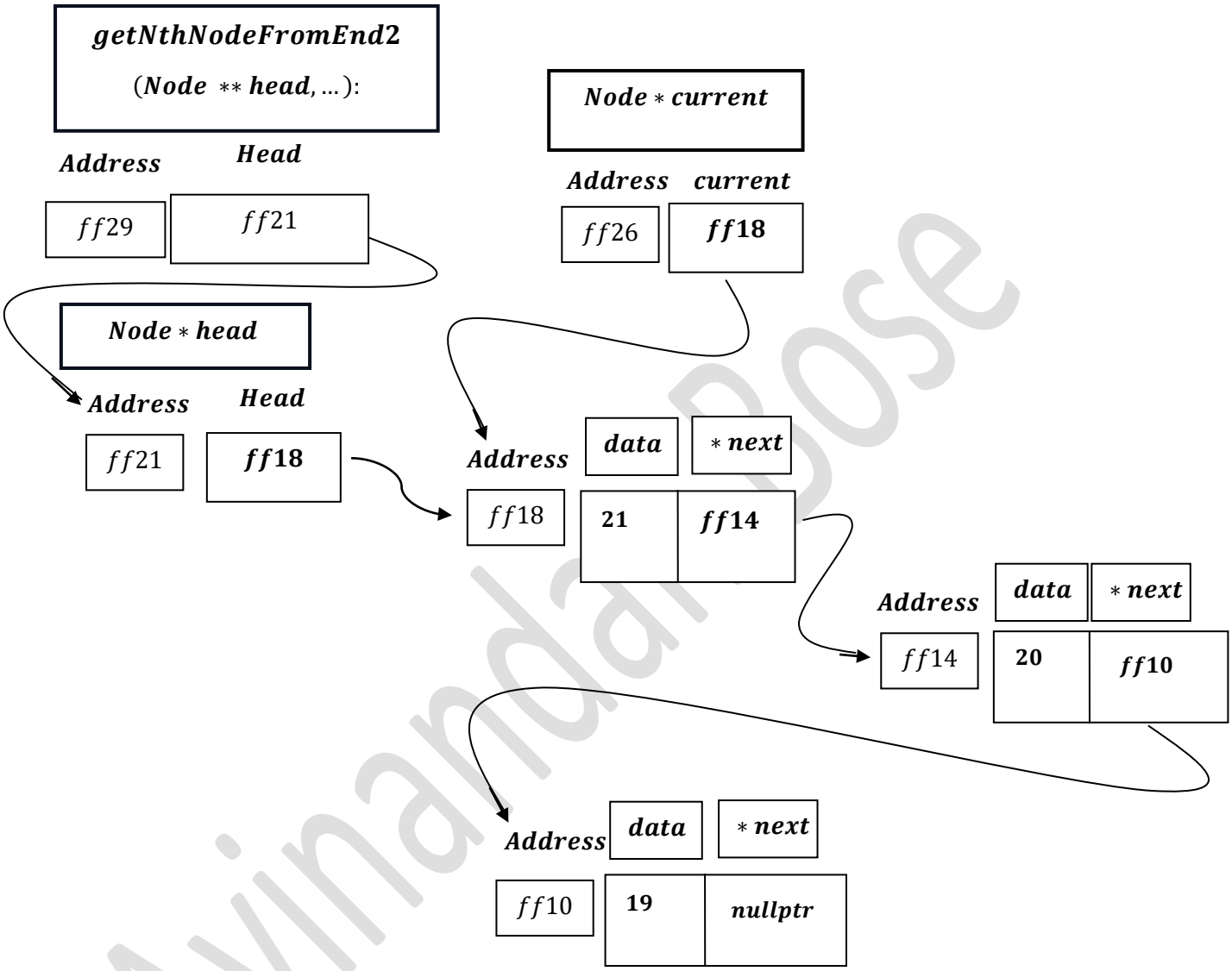
2) *Linked List is not Empty*:

Pictorially depicted as:



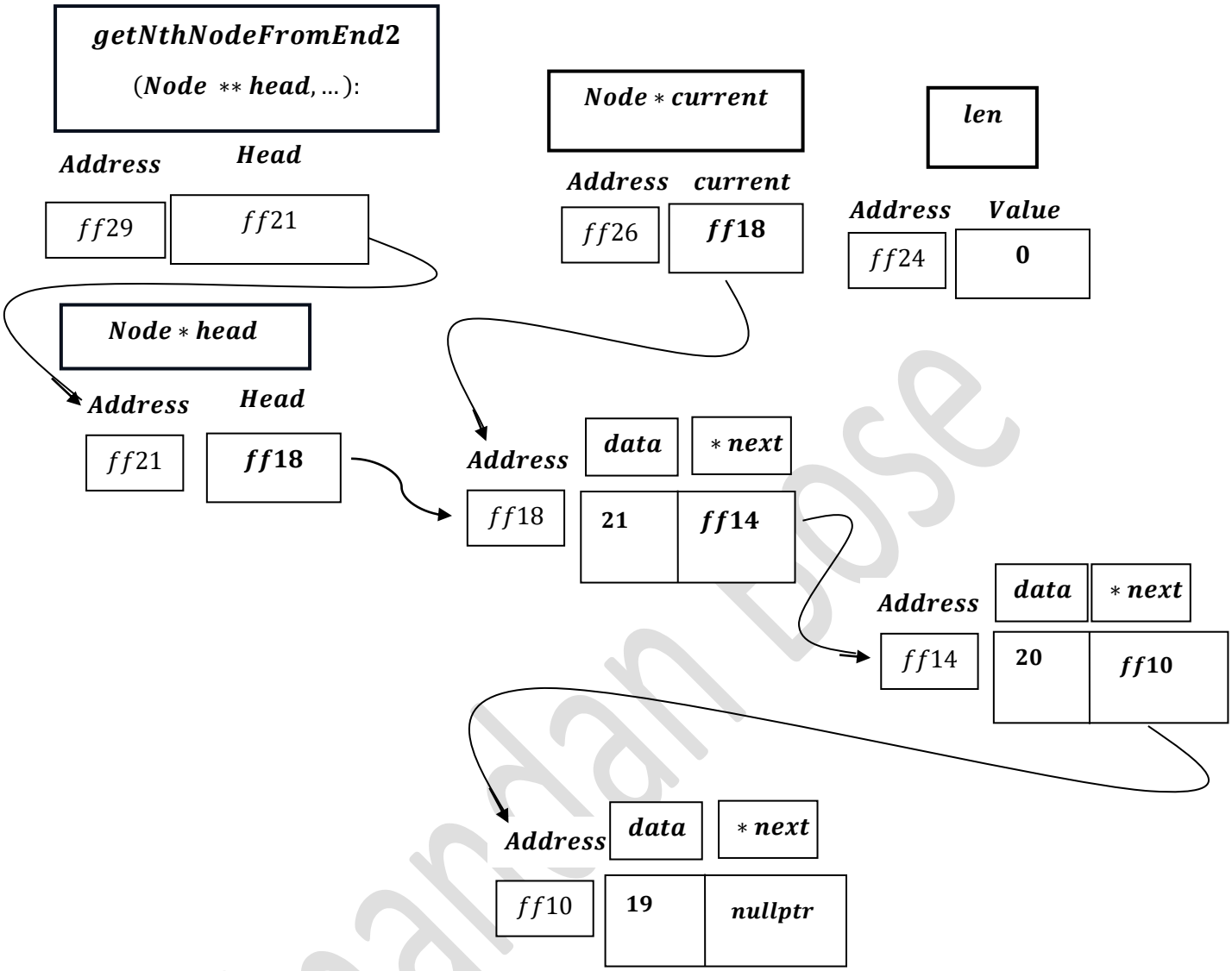
Avinandan Bose

```
Node *current;
current = *head;
```




```
    int len = 0;
```

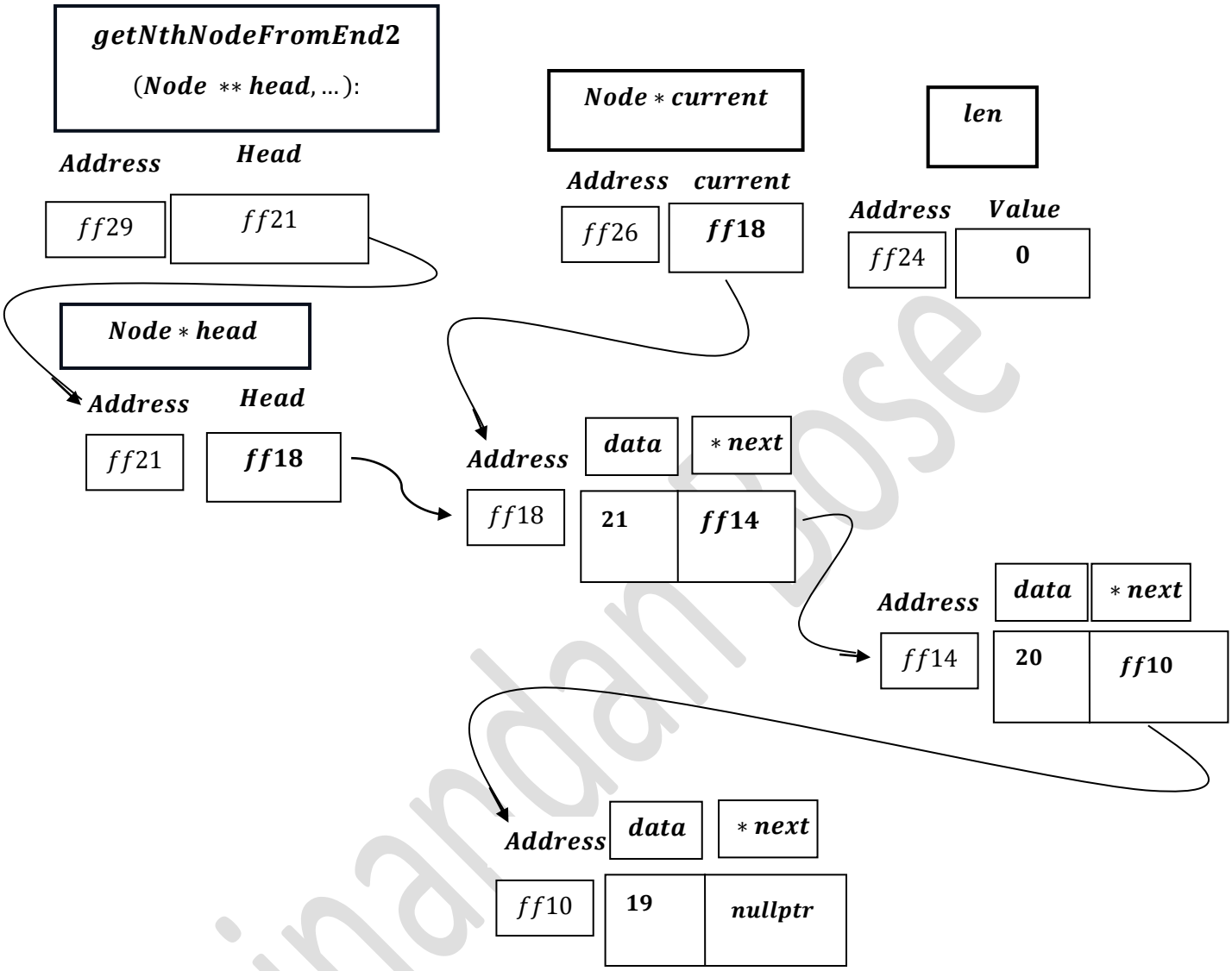
Avinandan Bose



***Now , we will count length of the linked list*: –**

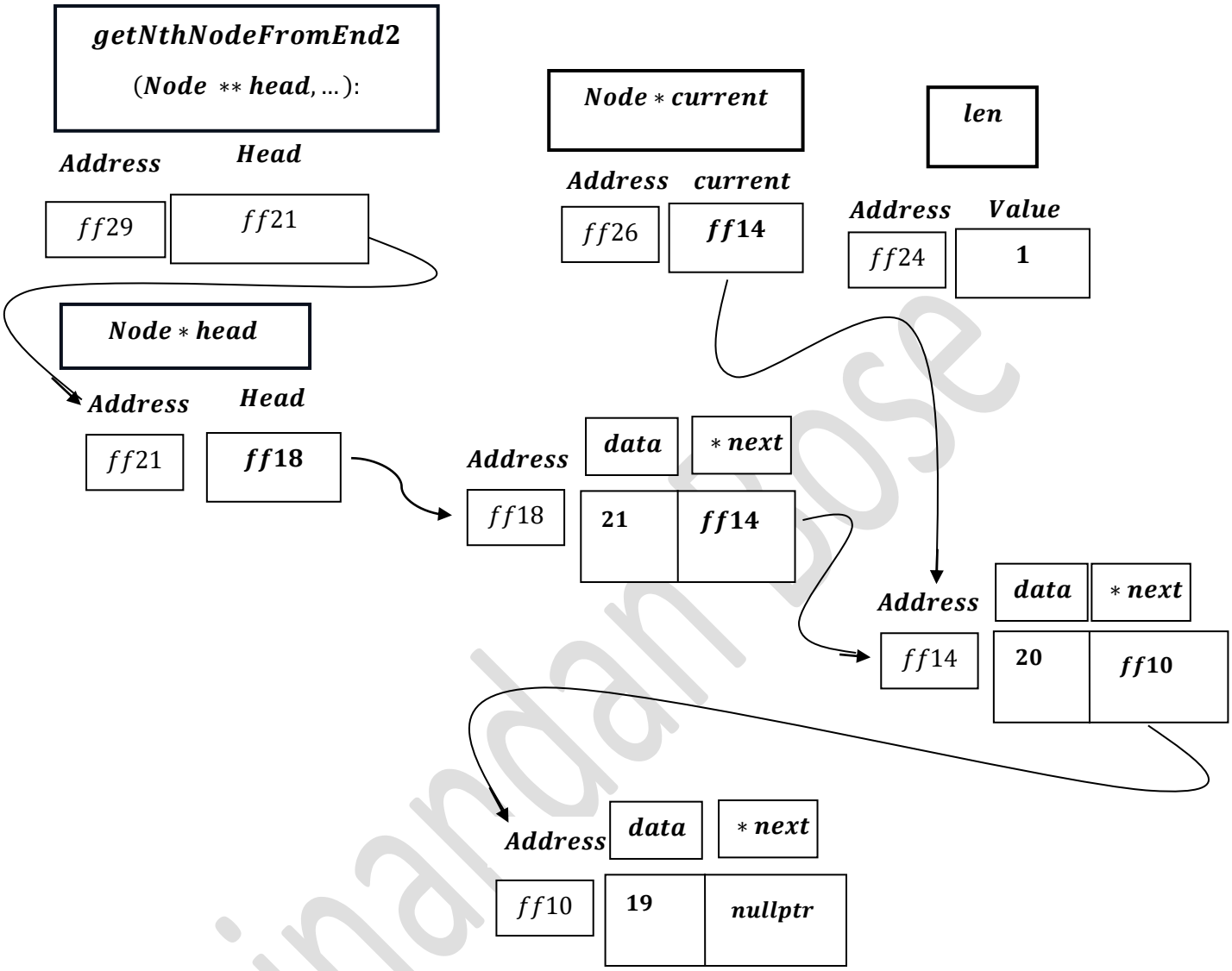
```
    while (current != nullptr)
    {
        current = current->next;
        len++;
    }
```

***We just recall the above picture* :**



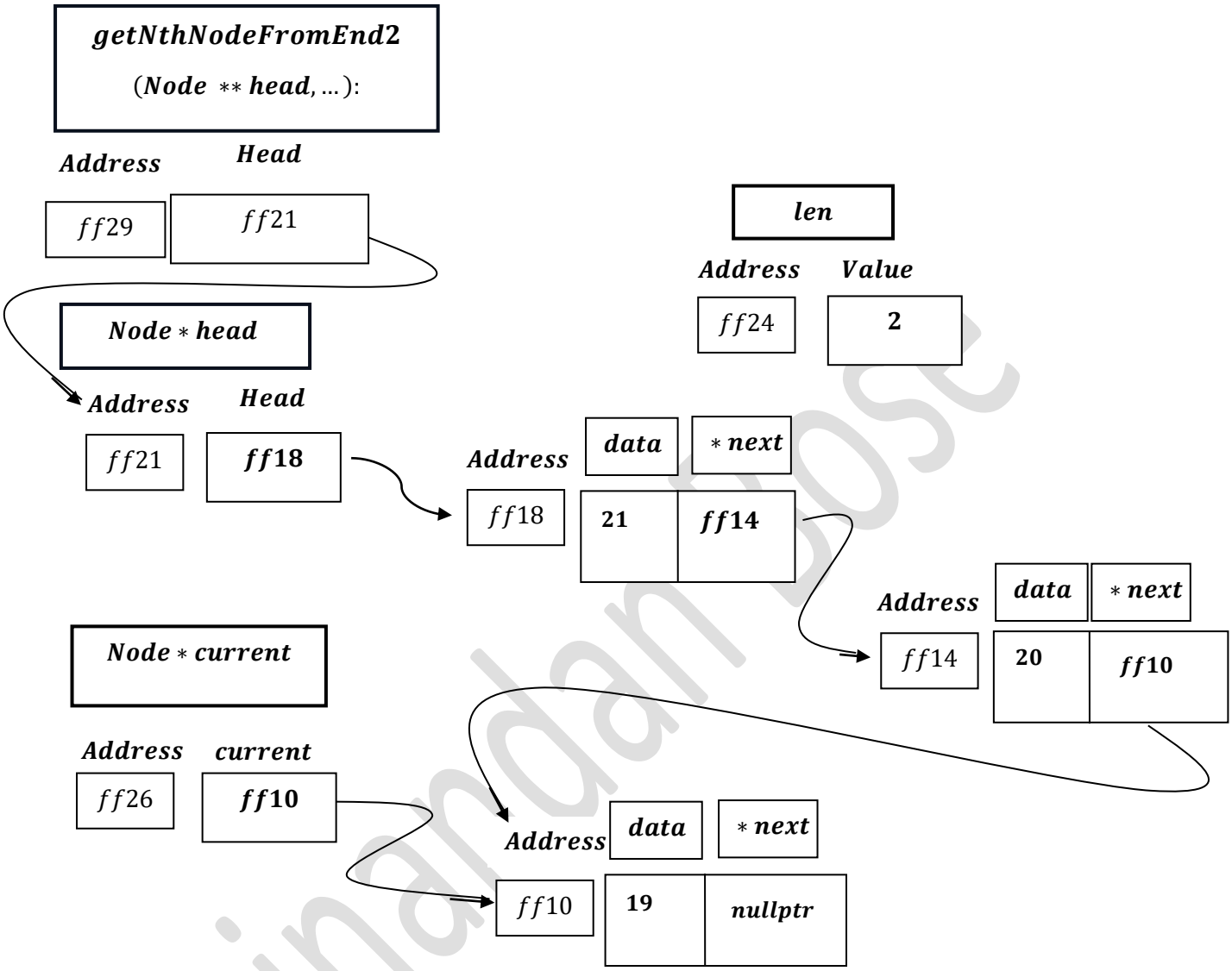

```
while(current != nullptr){
        current = current -> next; i.e. current = ff14;
        len ++; i.e. len = 0 + 1 = 1;
}
```

Avinandan Bose



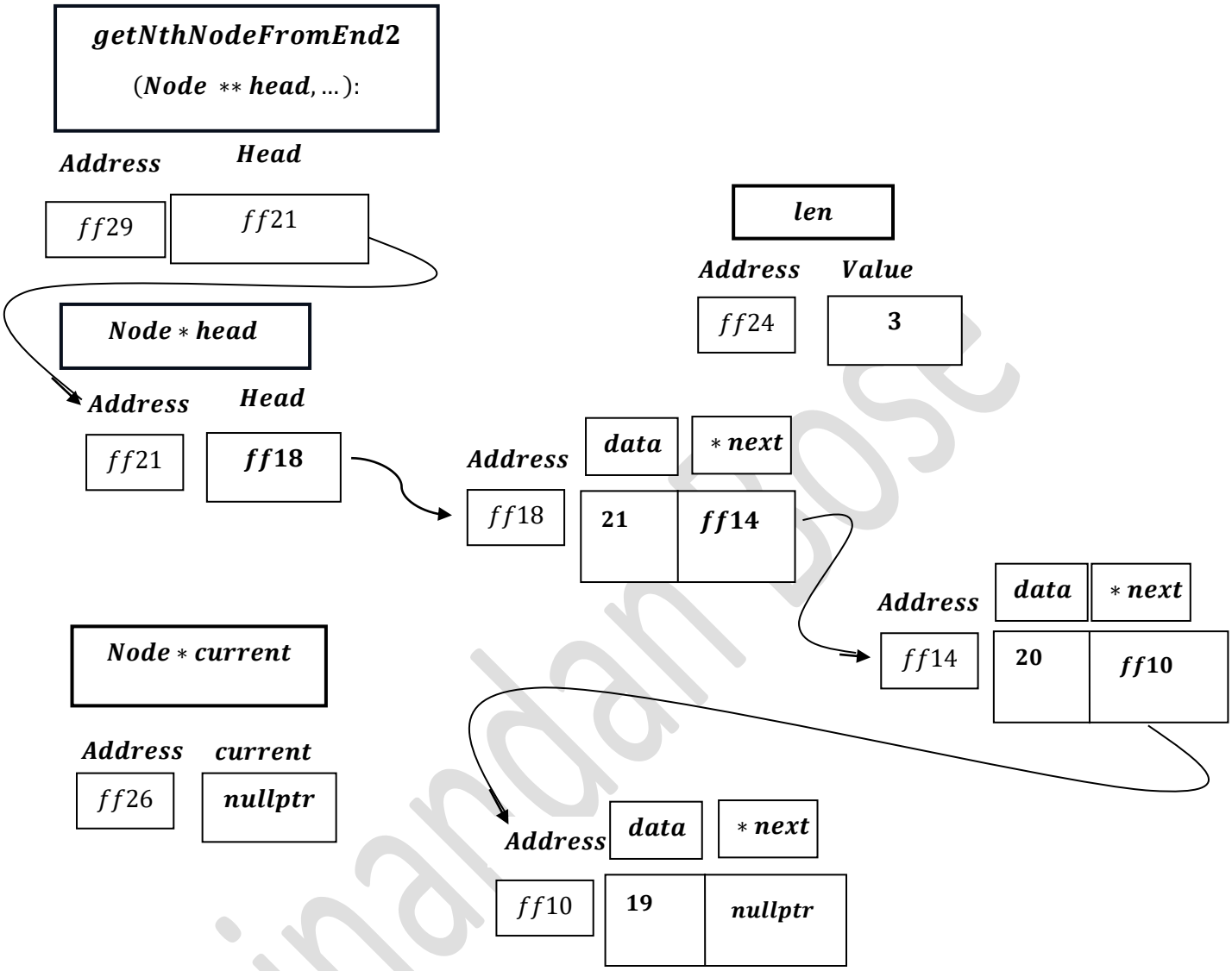

```
while(current != nullptr){
        current = current -> next; i.e. current = ff10;
        len ++; i.e. len = 1 + 1 = 2;
}
```

Avinandan Bose




```
while(current != nullptr){
        current = current -> next; i.e. current = nullptr;
        len ++; i.e. len = 2 + 1 = 3;
}
```

Avinandan Bose



Now, it checks whether user input `node` is less than `0` or `node` greater than or equal to `lengthOfLinkedList` as shown below:

```
if (node < 0 || node >= len)
    {
        cout << "Error: Invalid index." << endl;
        return -1;
    }
```

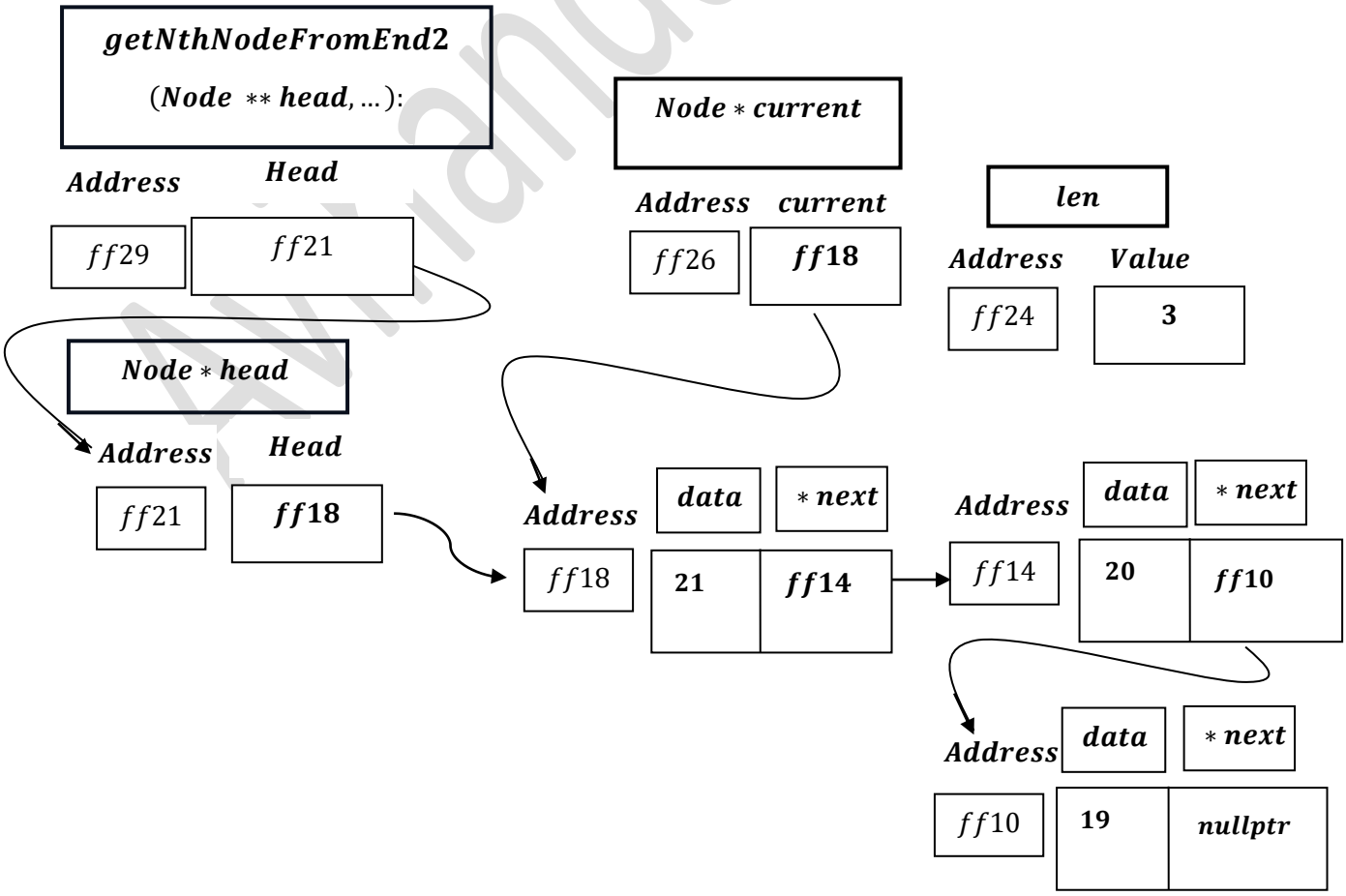
It outputs ``Error: Invalid index.`` and returns `−1`.

Now if user input node not less than 0 or `node` not greater than or equal to `lengthOfLinkedList` , then it will move to else part:

```
else
    {
        current = *head;
        for (int i = 0; i < len - node - 1; i++)
        {
            current = current->next;
        }
        return current->data;
    }
}
```

Now,

```
current = *head;
```



$Now,$

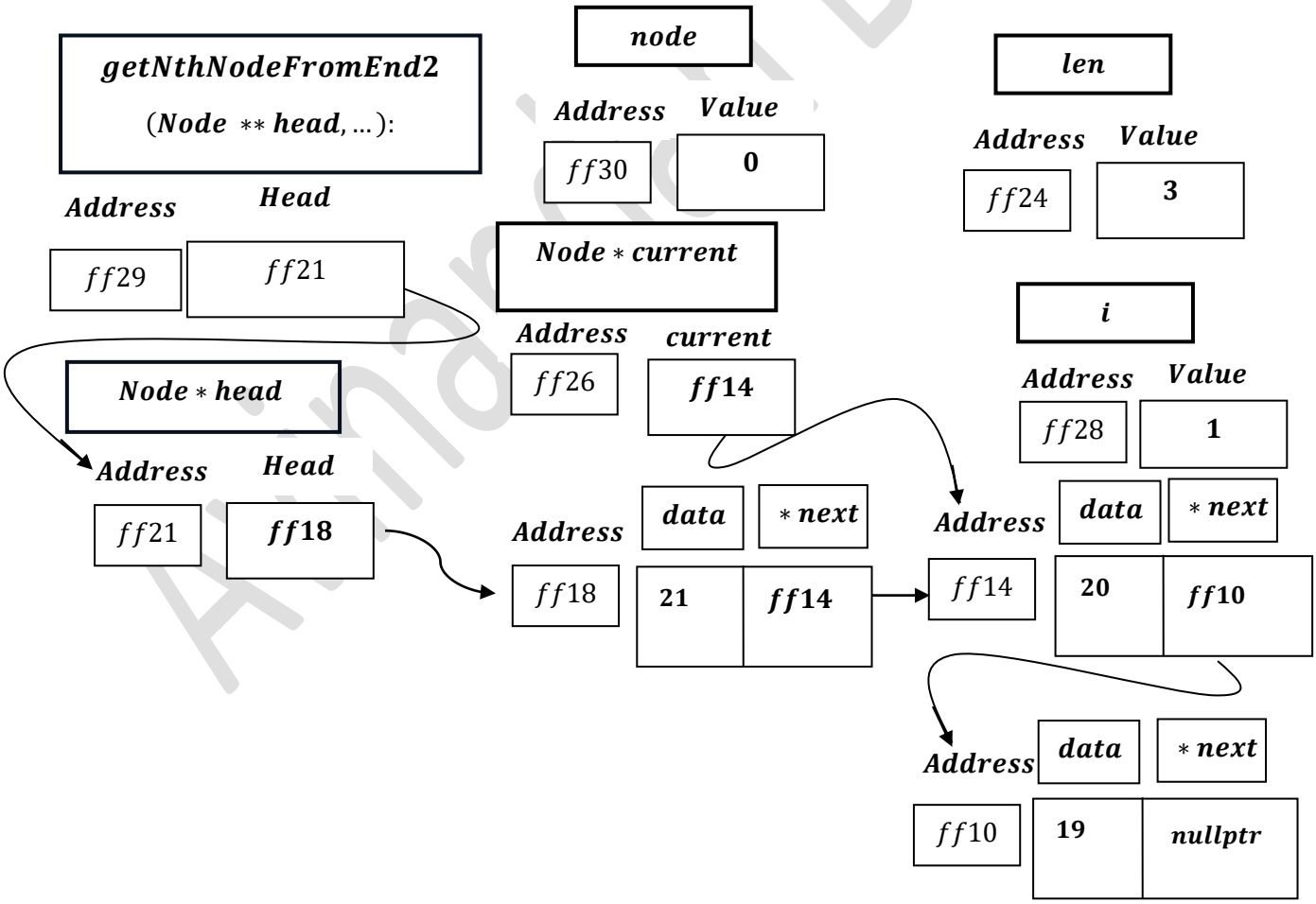
```
        for (int i = 0; i < len - node - 1; i++)
        {
            current = current->next;
        }
```

$Say, node = 0 (user\ input).$

$for\ i = 0\ and\ i < (len: 3 - node: 0 - 1)\ i.e.\ i < 2:$

$current = current \rightarrow next: ff14;$

$i = i + 1 = 0 + 1 = 1;$

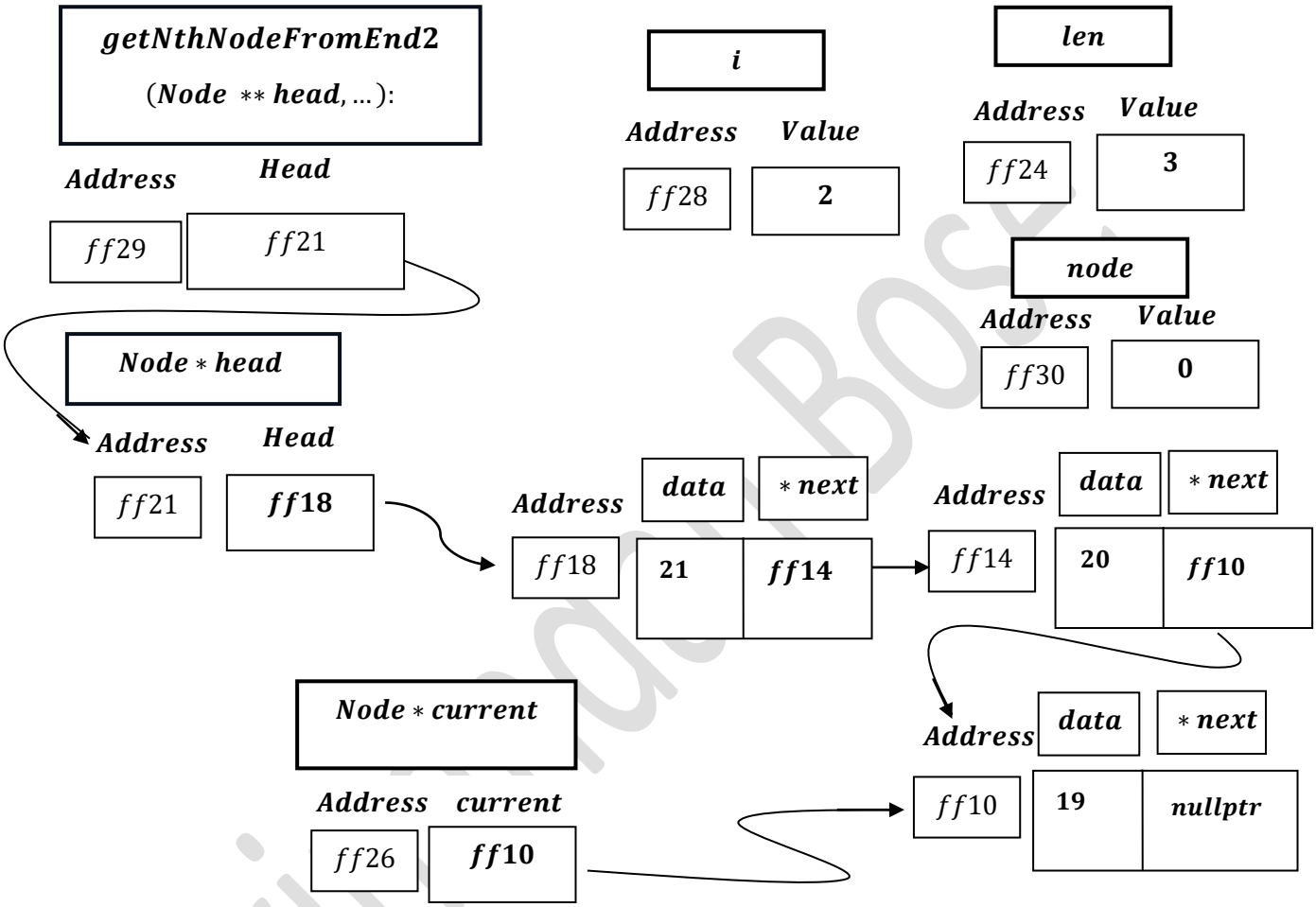


Avinandan Bose

$for\ i = 1\ and\ i < (len: 3 - node: 0 - 1)\ i.e.\ i < 2:$

$current = current \rightarrow next: ff10;$

$i = i + 1 = 1 + 1 = 2;$



$For\ next\ updation, for\ i = 2\ and\ \ i < 2\ , hence\ \ condition\ is\ false\ ,$

$hence\ loop\ exits.$

```
        return current->data;
```

$Now, it\ returns\ current \rightarrow data; i.e, data\ stored\ in\ 3rd\ Node\ i.e.\ 19\ .$

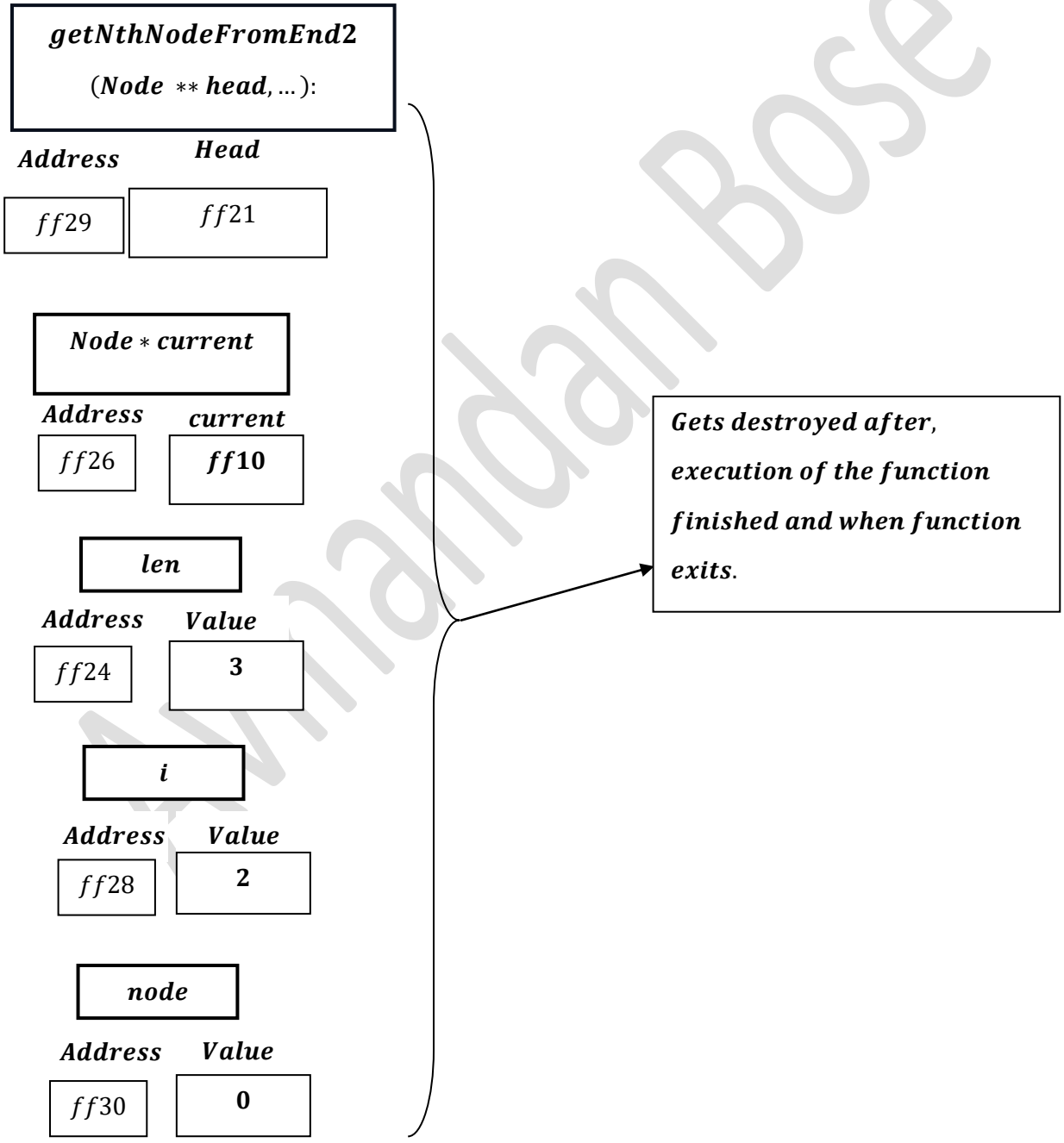
Thus we get Node from End i.e. if we enter node: 0 → It will give us n^{th} node's data.

if we enter node: 1 → It will give us $(n-1)^{th}$ node's data.

if we enter node: 2 → It will give us $(n-2)^{th}$ node's data.

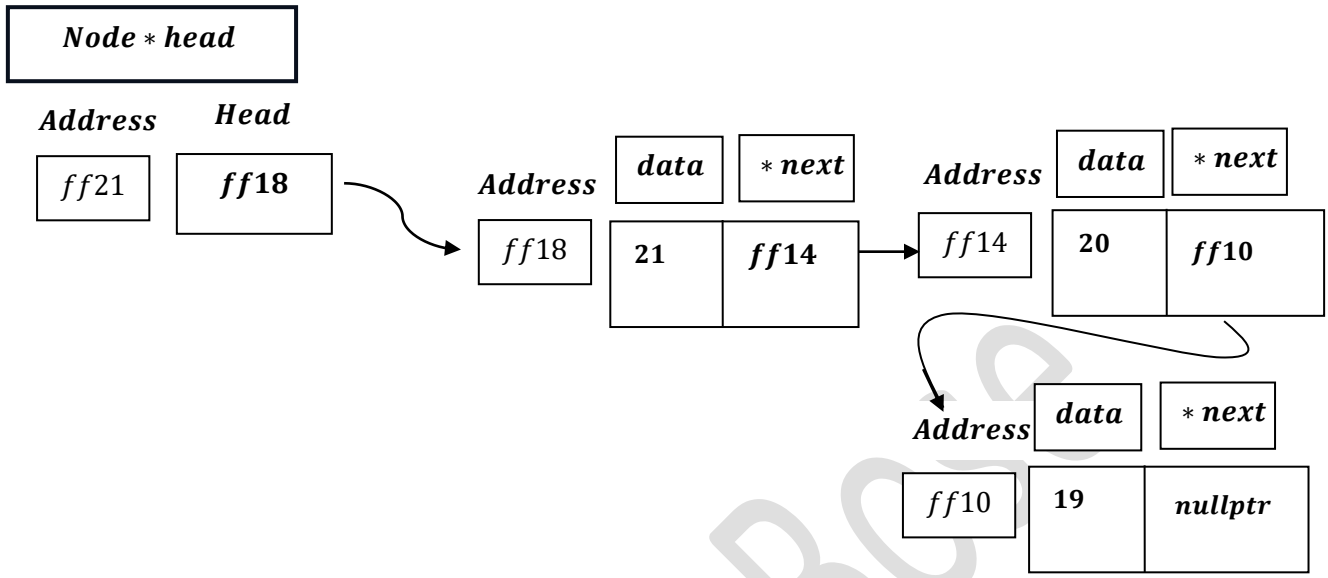
.... And it goes like this.

Now, as the function get finished, all temporary variable created earlier in process for the program will get destroyed.



Avinandan Bose

And we are left with:



Now, `getNthNodeFromEnd2` step by step explanation :

1. *Initialize Length and Current Pointer*:

- *An integer len is initialized to* 0 *to keep track of the length of the linked list.*
- *A pointer current is initialized to point to the head of the list.*

2. *Calculate the Length of the List*:

- *The function enters a loop to traverse the entire list to calculate its length.*
- *In each iteration, it moves the current pointer to the next node and increments len.*

3. *Check for Invalid Node Index*:

- *The function checks if the provided node index is less than* 0 *or greater than or equal to the length of the list.*

- *If the index is invalid, it prints an error message and returns* -1.

Avinandan Bose

4. *Reset Current Pointer*:

- *The current pointer is reset to point to the head of the list again.*

5. *Traverse to the* $(len - node - 1)th$ *Node*:

- *The function enters a loop to traverse the list until it reaches the* $(len - node - 1)th$ *node from the start.*

  $for\ (int\ i = 0;\ i < len - node - 1;\ i++)$: *This loop moves current to the desired node. The logic* $len - node - 1$ *is used because we are looking for the nth node from the end.*

- *This is equivalent to the nth node from the end of the list in a* $0-$*based index system.*

6. *Return Node Data*:

- *Once the desired node is reached, the function returns the data of the current node.*

Avinandan Bose

Time Complexity of getNthNodeFromEnd

```
int getNthNodeFromEnd(Node **head, int node)
{
    int len = 0;                                  O(1)
    Node *current;                                O(1)
    current = *head;                              O(1)
    while (current != nullptr)
    {
        current = current->next;                  runs n times.
        len++;                                    runs n times.
    }
    if (node < 0 || node >= len)                  checks condition in constant time.
    {
        cout << "Error: Invalid index." << endl;  runs 1 time i.e. O(1)
        return -1;
    }

    else
    {
        current = *head;                          takes O(1) time.
        for (int i = 0; i < len - node - 1; i++)
        {
            current = current->next;              if node is 0, loop runs `n − 1` times,
                                                  statement also runs `n − 1` times.
        }
        return current->data;                     takes O(1) time.
    }
}
```

By Avinandan Bose

$\underline{Worst\ Case}$

$\underline{1.When\ Node\ is\ negative\ and\ node\ is\ greater\ than\ length:}$

- $when\ `len = 0`\ , this\ assignment\ takes\ constant\ amount\ of\ time\ , O(1).$
- $Node * current\ ; declaration\ takes\ constant\ amount\ of\ time, O(1).$
- $current = * head; assignment\ takes\ constant\ amount\ of\ time\ , O(1).$
- $Calculation\ of\ length\ `len`\ takes\ `n`\ times\ also\ `current = current \rightarrow next`$ $assignment\ takes\ `n`\ times\ , hence\ it\ takes\ in\ total: n + n = 2n = O(n).$
- $As\ `node`\ is\ less\ than\ 0\ or\ `node`\ is\ greater\ than\ or\ equal\ to\ `len`\ , it\ will\ output$ $a\ statement\ and\ return - 1\ , which\ takes\ constant\ amount\ of\ time$ $, i.e. O(1).$

  $Hence\ total\ amount\ of\ time\ taken: -O(1) + O(1) + O(1) + O(n) + O(1) = O(n).$

$\underline{2.When\ `node`\ is\ entered : `0`}$

- $when\ `len = 0`\ , this\ assignment\ takes\ constant\ amount\ of\ time\ , O(1).$
- $Node * current\ ; declaration\ takes\ constant\ amount\ of\ time, O(1).$
- $current = * head; assignment\ takes\ constant\ amount\ of\ time\ , O(1).$
- $Calculation\ of\ length\ `len`\ takes\ `n`\ times\ also\ `current = current \rightarrow next`$ $assignment\ takes\ `n`\ times\ , hence\ it\ takes\ in\ total: n + n = 2n = O(n).$
- $Now\ , node\ is\ `0`\ , then\ it\ just\ checks\ that\ `node`\ less\ than\ `0`\ , which\ is\ false$ $this\ takes\ constant\ `c`\ amount\ of\ time\ which\ is\ `O(1)`\ .$
- $As\ `node`\ input\ is\ 0\ , therefore\ statement\ , `current = current \rightarrow next`\ ,$ $assignment\ takes\ `n - 0 - 1 = n - 1`\ times. As\ current\ already\ points\ to$ $head\ pointer\ or\ 1st\ node\ to\ start\ with\ and\ while\ loop\ starts\ it\ will\ get$ $assigned\ to\ address\ of\ 2nd\ node\ , then\ the\ 3rd\ node, \ldots, to\ address$ $of\ nth\ node. Hence\ 2nd\ to\ nth\ node\ i.e. loop\ runs\ n - 1\ times.$

  $Therefore\ , this\ results\ , O(n - 1) = O(n).$

Avinandan Bose

- $Now, it\ returns\ current \rightarrow data; which\ takes\ constant\ `c`\ amount\ of\ time,$ $which\ is: O(1).$

  $Hence\ total\ amount\ of\ time\ taken: -O(1) + O(1) + O(1) + O(n) + O(1) + O(n) + O(1) = O(n).$

$\underline{Average\ Case}$

$$\left(\begin{array}{c} uniform\ distribution\ of\ nodes - \\ distribution\ where\ all\ outcomes\ are\ equally\ likely. \end{array}\right)$$

$Uniform\ Distribution: This\ is\ a\ statistical\ distribution\ where\ each\ possible\ value\ of\ a\ random\ variable\ is\ equally\ likely. In\ the\ context\ of\ this\ function, it\ means\ that\ each\ possible\ value\ of\ node\ (from\ 1\ to\ the\ length\ of\ the\ list)\ is\ equally\ likely\ to\ be\ requested.$

$Average\ Case: When\ analyzing\ the\ average\ case\ performance\ of\ an\ algorithm, we\ consider\ the\ expected\ performance\ over\ all\ possible\ inputs, assuming\ a\ certain\ distribution\ of\ inputs. For\ the\ getNthNodeFromEnd\ function, if\ we\ assume\ a\ uniform\ distribution\ of\ node, it\ means\ that\ each\ node\ position\ from\ the\ end\ is\ equally\ likely\ to\ be\ requested.$

$Continuing\ with\ the\ calculation:$

$\rightarrow 1st\ we\ have\ to\ calculate\ the\ length\ of\ the\ linked\ list\ i.e.\ `n`, \Theta(n).$

$\rightarrow 2nd\ based\ on\ `node`\ input, where\ `n`\ is\ length\ of\ linked\ list, then:$

$When\ node\ is\ 0, loop\ runs\ `n-1`\ times.$
$When\ node\ is\ 1, loop\ runs\ `n-2`\ times.$
$When\ node\ is\ 2, loop\ runs\ `n-3`\ times.$
.
.
.
$When\ node\ is\ `n-2`, loop\ runs\ 1\ time.$
$When\ node\ is\ `n-1`, loop\ runs\ 0\ time.$

Avinandan Bose

$$Therefore, when\ node\ entered\ from \left[\sum_{i=0}^{n-1} i\right], loop\ runs\ from \left[\sum_{i=0}^{n-1} i\right] times.$$

$$\sum_{i=0}^{n-1} i = a_0 + \sum_{i=1}^{n-1} i = 0 + \frac{n(n-1)}{2} = \frac{n(n-1)}{2}.$$

$Now, when\ `node = n-1\ or\ len-1`, loop\ runs\ 0\ time\ but\ `current \rightarrow data`\ returns\ i.e.\ the\ first\ node.$

$Hence, we\ have, \frac{n(n-1)}{2} + 1. Here\ 1\ for\ finding\ the\ first\ node.$

$Now, If\ there\ is\ `n`\ no.\ of\ nodes\ there\ we\ find\ data\ at\ `n`\ positions\ equally\ likely\ is:$

$For\ position\ 1, the\ probability\ of\ finding\ data\ is: \frac{1}{n}.$

$For\ position\ 2, the\ probability\ of\ finding\ data\ is: \frac{1}{n}.$

.

.

$For\ position\ n, the\ probability\ of\ finding\ data\ is: \frac{1}{n}.$

$Hence\ probability\ of\ finding\ data\ will\ always\ remain: \frac{1}{n}, equally\ likely.$

$\therefore\ Average\ Time\ Complexity\ is: probability\ of\ finding\ data \times (number\ of\ possible\ iterations + constant\ time\ complexities\ for\ left\ over\ positions).$

$$= \frac{1}{n} \times \left(\frac{n(n-1)}{2} + 1\right)$$

Avinandan Bose

$$= \frac{n-1}{2} + \frac{1}{n}$$

Appplying Big Theta in the equation:

$$= \Theta\left(\frac{n-1}{2}\right) + \Theta\left(\frac{1}{n}\right)$$

Now, taking out the dominant term of numerator and denominator:

$$= \Theta\left(\frac{n}{2}\right) + \Theta\left(\frac{1}{n}\right)$$

$$= \frac{1}{2} \times \Theta(n) + \Theta\left(\frac{1}{n}\right)$$

Now removing the constant `$\frac{1}{2}$` ***we get***:

$$= \Theta(n) + \Theta\left(\frac{1}{n}\right)$$

When combining two Big Theta Θ **terms , largest growth rate is the result**:

$\rightarrow$ $\Theta(n)$ ***grows linearly with*** `$n$` .

$\rightarrow$ $\Theta\left(\frac{1}{n}\right)$ ***decreases to a constant , i.e. when*** `$n$` ***grows larger ,*** $\frac{1}{n}$ ***approaches to*** 0.

This we can prove by limit and continuity also:

Avinandan Bose

$$\lim_{n\to\infty}\left(\frac{1}{n}\right) \Rightarrow \frac{1}{\infty} = 0 \left[\frac{c}{\infty} = 0\text{, where } c \text{ is constant}\right].$$

Hence we get `$\Theta(n)$` for all positions.

But as length already calculated earlier i.e. $\Theta(n)$, now calculation for all possibility to get `n` th positions equally likely doesnot actually matters, hence, either from calculation of length of node, we can say directly that Average Case is $\Theta(n)$, or,

Total complexity: $\Theta(n) + \Theta(n) = \Theta(n)$.

Note, by relation: $\Omega \leq \Theta \leq O$, *we get*:

$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$

$\Rightarrow c_1 \times 1 \leq n + \frac{n-1}{2} + \frac{1}{n} \leq c_2 \times n$

Alternatively,

→ *1st we have to calculate the length of the linked list i.e. `n`, $\Theta(n)$.*

For position 1, the probability of finding data is: $\frac{1}{n}$.

For position 2, the probability of finding data is: $\frac{1}{n}$.

.

.

Avinandan Bose

$$For\ position\ N, the\ probability\ of\ finding\ data\ is: \frac{1}{n}.$$

$$Hence\ probability\ of\ finding\ data\ always\ remain\ \frac{1}{n}, for\ position\ 1\ to\ n.$$

$$Average\ case\ complexity:\ Probability\ of\ finding\ data\ \times \begin{pmatrix} Summation \\ of\ number\ of \\ Position. \end{pmatrix}$$

$$= \frac{1}{n} \times \left(\sum_{i=1}^{n} i \right)$$

$$= \frac{1}{n} \times \left(\frac{n(n+1)}{2} \right)$$

$$= \frac{n+1}{2}$$

$$or, \frac{n}{2} + \frac{1}{2}$$

$$Appplying\ Big\ Theta\ in\ the\ equation:$$

$$= \Theta\left(\frac{n}{2}\right) + \Theta\left(\frac{1}{2}\right)$$

$$Now\ removing\ the\ constant\ `\frac{1}{2}`\ we\ get:$$

$$= \Theta(n) + \Theta(1)$$

Avinandan Bose

$= \Theta(n)$.

$Total\ complexity: \Theta(n) + \Theta(n) = \Theta(n)$.

$Note\ , by\ relation: \Omega \leq \Theta \leq O\ , we\ get:$

$\Longrightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$

$\Longrightarrow c_1 \times 1 \leq n + \frac{n+1}{2} \leq c_2 \times n$

Singly Linked List	*Cases*	*Time Complexity*
Get Nth Node From End2	1. *Best Case*	$\Omega(n)$ [*As, length is calculated first hence Worst Case = Best Case , here*]
	2. *Worst Case*	$O(n)$
	3. *Average Case*	$\Theta(n)$

****

Avinandan Bose